Driver Mobile Phone Use Detection: State-of-the-Art Review and a Lightweight Knowledge Driven Multimodal Framework

RIF Grant No. 23081

PROGRESS REPORT #2

Proposed DT Approach

Presented to: Dr. Fatma Outay

Date: August 2026

4. Research Contribution and Proposed Approach

4.1. Research Contribution

A closer examination of existing approaches to driver mobile-phone-use detection reveals a set of recurring challenges across modeling paradigms. Deep-learning techniques are effective at learning discriminative visual features, yet they typically depend on large labeled corpora and substantial computational resources, which limits their suitability for real-time in-vehicle deployment. Attention-based models help emphasize relevant spatial and temporal cues, but their performance degrades when training data are scarce or imbalanced, and their decisions remain difficult to interpret despite recent progress in explainable AI. Temporal models, including recurrent networks and transformers, capture sequential patterns in driver behavior but are sensitive to noisy inputs and often struggle to model long-range dependencies under naturalistic conditions. Ensemble strategies improve robustness by combining multiple learners, but at the cost of greater system complexity, longer inference times, and harder optimization and maintenance. Collectively, these observations expose the persistent trade-offs among accuracy, efficiency, and interpretability that continue to hinder the practical deployment of vision-based distraction-detection systems.

Our literature review indicates that the underlying research problem is not purely a sensing challenge; it also encompasses contextual and human-centered dimensions. No single modality can reliably capture driver mobile-phone use under real-world conditions, and each approach offers only an incomplete view constrained by tight trade-offs among cost, practicality, accuracy, privacy, scalability, and robustness. Moreover, although many studies report strong performance metrics (e.g. high recognition accuracy or F1 scores) the limited real-world deployment of these systems raises important questions about their practical applicability. Papatheocharous et al. (2023) identify several contributing factors. First, most research efforts prioritize novel methods, theoretical models, and prototypes evaluated for algorithmic performance rather than designed as deployable solutions. Second, these studies typically rely on publicly available or small, custom-collected datasets that are highly homogeneous; when such datasets are split into training and validation subsets, models can achieve very high accuracy, which explains the frequently reported 90–100% figures in image-based distracted-driving detection. Third, the conditions under which models are trained and validated do not reflect the complexity of real-world driving: variations in driver behavior, posture, and appearance; differences in camera angles; dynamic lighting; glare and reflections; and nighttime driving all introduce variability that controlled datasets rarely capture. Finally, models are usually evaluated in laboratory or standardized conditions designed to maximize performance metrics for publication, further limiting external validity and practical readiness.

Fu et al. (2024) argue that research on driver mobile-phone use should be driven not only by technical innovation but also by a commitment to inclusivity, so that advances in driver-monitoring technology yield meaningful and equitable safety improvements across diverse driver populations, vehicle types, and driving contexts. In particular, because mobile-phone-based monitoring operates independently of vehicle-embedded sensors and actuators, it provides a practical means of retrofitting older vehicles with driver-monitoring capabilities.

We argue that using the mobile phone itself as a sensing platform offers a natural and intuitive means of detecting driver phone use, since the device is simultaneously the source of distraction and a direct record of user interactions. Unlike vision-based systems, which require cameras and are sensitive to lighting, occlusion, and viewpoint, or wearable solutions, which can be intrusive and limit user acceptance, phone-centric approaches leverage the device's existing sensors to capture motion, usage patterns, and contextual cues. This alignment between sensing modality and target behavior enables timely, accurate, and scalable detection while reducing deployment complexity and cost, making it particularly well-suited to retrofits and broad adoption across diverse driving environments.

On this basis, we propose a lightweight, knowledge-driven multimodal decision tree (DT) for deterministic detection and risk-severity assessment of mobile-phone-related driver distraction. The DT fuses three complementary data streams (OBD-II vehicle kinematics, smartphone inertial sensors, and application-level behavioral telemetry) and exposes every decision rule for audit, requires no labeled training corpus, and runs in O(1) time per five-second observation window, making it deployable on resource-constrained OBD-II dongles and mid-range Android smartphones alike. The proposed system, designated LWMS (Lightweight Multimodal System), shifts the detection paradigm from visual approximation of posture to direct behavioral evidence of phone interaction, thereby providing a more accurate and actionable representation of distracted driving than camera-based alternatives.

The principal contribution of this work is the design and implementation of LWMS. In contrast to the dominant camera-based paradigm, which infers distraction from visual posture in video frames and demands computationally intensive deep-learning models alongside dedicated camera infrastructure, LWMS introduces a fundamentally different detection philosophy grounded in confirmed behavioral events and contextual vehicle dynamics. The specific contributions are as follows:

1) Behavior-driven detection of actual phone-interaction events. Whereas camera-based systems approximate distraction by classifying visual posture (head pose, hand position), LWMS directly detects explicit phone-usage events—keyboard input, social-

media activity, and call interactions—through on-device monitoring of application-level and input-level signals. This shift from visual inference to behavioral confirmation is designed to reduce false positives arising from posture misclassification and to ensure that alerts are issued only when genuine phone interaction is occurring.

2) Multimodal fusion of vehicle telemetry and smartphone interaction data. Rather than relying on a single modality, LWMS integrates real-time OBD-II vehicle telemetry (notably vehicle speed) with concurrent phone-interaction events captured at the application level. This fusion enables contextual risk assessment that neither source can provide in isolation: vehicle dynamics establish the driving context (e.g., in motion versus stationary), while phone signals identify the behavioral event. The combination distinguishes, for instance, a driver checking a social-media feed at highway speed from the same action performed while parked, supporting risk-leveled assessment rather than binary classification.

3) Risk-leveled alert generation beyond binary classification. Most existing systems output either a binary distracted/not-distracted label or a coarse posture category (e.g., texting, talking, reaching). LWMS introduces a graded alert mechanism that classifies detected events into discrete risk levels reflecting the contextual severity of the phone interaction relative to the current driving state. This finer-grained output enables more nuanced driver feedback and supports differentiated intervention strategies, improving practical utility and user acceptance.

4) Interpretable, decision-tree-based reasoning. Although convolutional and recurrent neural networks achieve strong classification performance in camera-based detection, their opacity poses significant barriers to deployment in safety-critical domains where regulatory and user-trust requirements demand explainability. LWMS employs a lightweight decision tree that produces human-readable rules, ensuring that every alert can be traced to a transparent chain of sensor-level evidence. This interpretability facilitates regulatory compliance, user trust, and iterative system refinement.

5) Camera-free, privacy-preserving architecture. Camera-based monitoring raises substantial privacy concerns, as continuous in-cabin recording captures drivers' and passengers' faces, conversations, and personal belongings. LWMS is camera-free by design: it relies exclusively on OBD-II telemetry and on-device phone-interaction signals, neither of which captures visual or personally identifiable information. This architectural choice reduces privacy risk and simultaneously removes the environmental limitations that degrade camera performance, such as poor lighting, occlusion, and adverse weather.

6) Lightweight, on-device deployment with no additional hardware. LWMS operates entirely as a background Android service, performing all sensing, inference, and alert

generation on the smartphone without offloading computation to external servers or requiring supplementary sensors beyond a standard OBD-II adapter. This on-device architecture ensures low latency, eliminates dependence on network connectivity, and preserves data locality. Additionally, the system is compatible with any vehicle equipped with a standard OBD-II port, covering the vast majority of vehicles manufactured after 2008,without requiring vehicle-specific calibration, dedicated camera installation, or proprietary hardware, making it immediately deployable at negligible cost.

7) Context-aware exemptions and actionable driver feedback. The framework incorporates context-aware exemptions—most notably a navigation-app exemption that respects modern ergonomic standards under which dashboard-mounted navigation is legally and cognitively permissible. In addition, LWMS is paired with a mobile application that generates detailed statistical reports, helping drivers understand when, where, and how often they become distracted. These reports transform raw detection data into actionable insights and empower drivers to make informed decisions about their phone-use habits.

We provide empirical evidence, based on real-world driving data, that this multimodal behavior-driven approach achieves effective detection performance while maintaining the computational efficiency required for continuous on-device operation. Collectively, these contributions establish a privacy-preserving, behavior-aware, multimodal, and deployment-oriented alternative to camera-dominant distraction-detection systems, advancing the state of the art toward scalable real-world monitoring of mobile-phone-induced driver distraction.

The proposed approach eliminates the need for cameras entirely. Instead, it leverages two data sources that are readily available in modern vehicles and smartphones: (i) the vehicle's On-Board Diagnostics (OBD-II) port, which provides real-time speed, and (ii) the driver's smartphone built-in inertial sensors (accelerometer and gyroscope) combined with application-level behavioral indicators such as typing activity, scrolling events, phone-call state, and foreground application category. The solution is implemented as a mobile application embedded in the driver's phone, making it immediately deployable without any additional hardware installation beyond a standard Bluetooth OBD-II adapter.

A critical design requirement is the avoidance of false alarms. The system must distinguish between legitimate phone usage, such as using a navigation application while driving or using the phone while the vehicle is stationary, and genuinely dangerous distraction patterns. Furthermore, not all distractions carry the same risk: texting while driving demands sustained visual, manual, and cognitive attention and is more dangerous than scrolling a webpage, which in turn differs from taking a phone call, which primarily imposes cognitive load with periodic manual demands. To this end, we propose a decision-tree-based approach grounded in expert-defined rules and thresholds that

explicitly classify four distraction types (phone calls, texting, social-media interaction, and scrolling) and produce three levels of alert severity (Low–Moderate–High), calibrated according to vehicle speed and the nature of the phone activity.

The remainder of this section details the proposed methodology, covering the system architecture (Section 4.2) and the decision-tree approach (Section 4.3).

4.2. System Architecture

As illustrated in Figure 1, the proposed system follows a layered architecture comprising four principal components: data acquisition, feature extraction, the decision engine, and alert output.

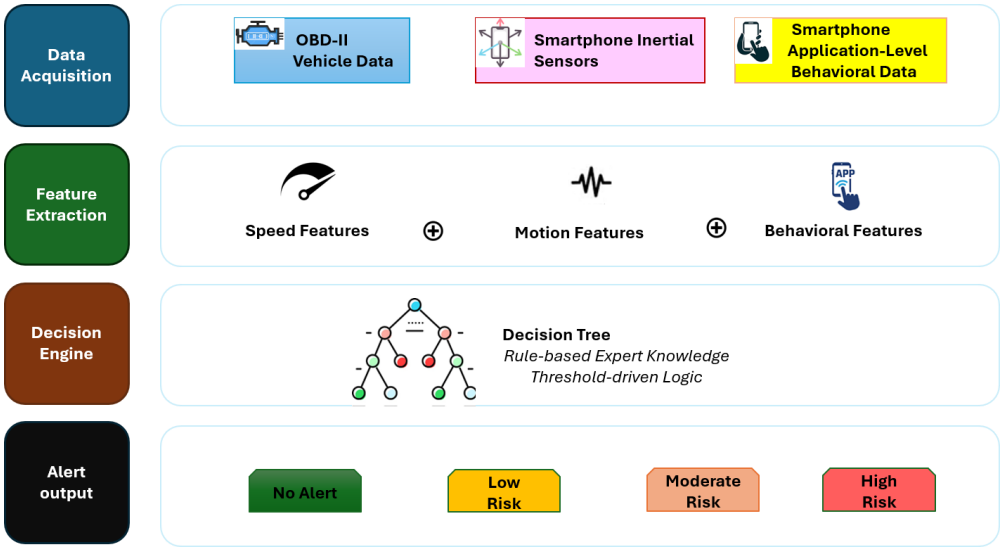


Figure 1. System Architecture

4.2.1. Data Acquisition Layer

The data acquisition layer collects raw signals from three distinct sources, all of which are accessible through the driver's smartphone:

• **OBD-II Vehicle Data:** A standard Bluetooth OBD-II adapter, connected to the vehicle's diagnostic port, transmits real-time vehicle speed to the mobile application via the ELM327 protocol. The primary signal of interest is the instantaneous vehicle speed $v(t)$, sampled at approximately 1 Hz (the standard OBD-II query rate). This signal is critical for determining whether the vehicle is in motion and for calibrating the risk level of any detected distraction.

• **Smartphone Inertial Sensors:** The phone's built-in three-axis accelerometer and three-axis gyroscope provide continuous motion data at sampling rates of 50–100 Hz. These sensors capture the characteristic motion patterns associated with phone handling (e.g. lifting the device to the ear for a call, tapping the screen, swiping, or typing) which

are clearly distinguishable from the vibrations and accelerations transmitted through the vehicle chassis when the phone is mounted in a cradle or resting on a surface.

• **Application-Level Behavioral Data:** The Android and iOS operating systems expose APIs that allow the monitoring application to detect: (a) the category of the foreground application (navigation, messaging, scrolling, social media, calling, or other); (b) touch-event counts and rates; (c) scroll-event detection; (d) keyboard activation (typing detection); and (e) phone-call state (idle, ringing, off-hook). These behavioral indicators provide direct evidence of phone interaction and its nature.

4.2.2. Feature Extraction

4.2.2.1. Sliding Window Segmentation

Raw sensor signals are segmented using a sliding window of duration W = 5 s with a stride of S = 2.5 s (i.e., 50% overlap, yielding a new observation every 2.5 s). The choice of window duration reflects a trade-off between two competing requirements: a sufficiently long window to estimate statistical features reliably, and a sufficiently short window to detect rapid changes in driver behavior. A 5-second window at 50 Hz yields 250 samples per inertial axis; with a 2.5-second stride (50% overlap), the framework produces a new classification every 2.5 s while retaining enough observations for stable parametric estimation

Extracted Feature Vector

As illustrated in Figure 2, our feature extraction pipeline produces a 9-dimensional feature vector $\vec{F}$, comprising a mix of continuous and categorical variables, and organized into three clusters corresponding to the three data acquisition sources. In addition, we derive a composite binary feature vector $\vec{C}$, whose elements are computed from a subset of the components of $\vec{F}$.

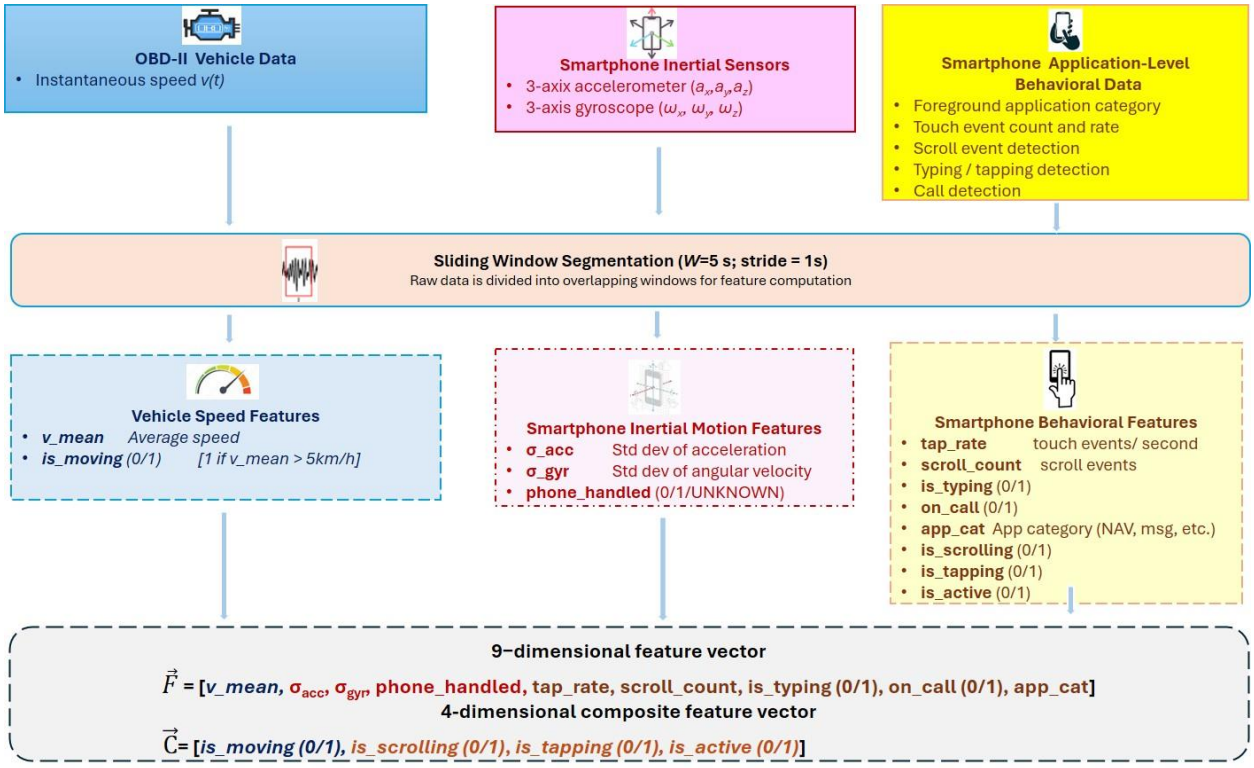


Figure 2. Multi-modal Feature extraction pipeline

4.2.2.1 OBD-II Vehicle Kinematics Features

From the OBD-II speed signal $v(t)$ over the measurement window $[t-W,t]$, we extract:

- $\boldsymbol{v_mean}$: Mean vehicle speed, which is the arithmetic mean of the Vehicle Speed PID (0x0D) sampled at 10 Hz over the five-second window, yielding 50 samples per observation

$$v_mean = \frac{1}{N}\sum_{i=1}^{N} v\,(t_i)$$

where N is the number of OBD-II samples within the window.

From the above, we derive the composite feature:

$\boldsymbol{is_moving}$: A binary indicator, equals 1 if [$v_{\boldsymbol{mean}} >$ V_{min}] where V_{min} = 5 km/h, and 0 otherwise. This threshold accounts for GPS/OBD-II measurement noise and minor vehicle creep, ensuring that the vehicle is genuinely in motion before any alert is triggered. The value of 5 km/h is chosen to suppress the low-speed jitter that occurs during engine idle and stop-and-go traffic at signalized intersections. We emphasize that this is a system-design threshold, not a universally established road-safety threshold.

4.2.2.2 Smartphone Inertial Features

We rely on the smartphone's built-in tri-axial gyroscope and accelerometer, sampled at the platform's native rate (typically 50–100 Hz). Both sensors are chosen because they capture complementary aspects of device motion: the gyroscope measures angular velocity, which is directly responsive to rotational manipulations such as tilting and reorientation, while the accelerometer measures proper acceleration (i.e., specific force), which responds to translational movements such as lifting, sliding, or tapping. Mobile phone handling produces inertial fluctuations in acceleration magnitude that are significantly larger than those produced by vehicle vibration alone, while rotational movements associated with lifting or tilting the phone produce elevated gyroscope variance.

We rely on the smartphone's built-in tri-axial gyroscope and accelerometer, sampled at the platform's native rate (typically 50–100 Hz). Both sensors are chosen because they capture complementary aspects of device motion: the gyroscope measures angular velocity, which is directly responsive to rotational manipulations such as tilting and reorientation, while the accelerometer measures proper acceleration (i.e., specific force), which responds to translational movements such as lifting, sliding, or tapping. Mobile phone handling produces inertial fluctuations in acceleration magnitude that are significantly larger than those produced by vehicle vibration alone, while rotational movements associated with lifting or tilting the phone produce elevated gyroscope variance

Raw readings are partitioned into non-overlapping windows of duration W = 5 s. Within each window of N samples, a unified feature-extraction procedure is applied identically to both sensors. For each observation window, the standard deviation of the tri-axial accelerometer and gyroscope signals is computed to characterize the magnitude of translational and rotational fluctuations. First, the instantaneous signal magnitude vector (SMV) is computed for each sample index i. More precisely, from the 3-axis accelerometer $\boldsymbol{a}_i = (a_{xi}, a_{yi}, a_{zi})$ and 3-axis gyroscope $\boldsymbol{\omega}_i = (\omega_{xi}, \omega_{yi}, \omega_{zi})$ samples, streamed at 50 HZ, we compute the following:

• $\boldsymbol{\sigma}_{\mathbf{acc}}$: Standard deviation of the acceleration magnitude, $\|\boldsymbol{a}_i\| = \sqrt{a_{xi}^2 + a_{yi}^2 + a_{zi}^2}$, defined as:

$$\sigma_{acc} = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(\|\boldsymbol{a}_i\| - \bar{a})^2} \qquad (1)$$

With $\bar{a} = \frac{1}{N}\sum_{i=1}^{N}\|\boldsymbol{a}_i\|$

• $\boldsymbol{\sigma_{gyr}}$: Standard deviation of the angular velocity magnitude, $\|\boldsymbol{\omega_i}\| = \sqrt{\omega_{xi}^2 + \omega_{yi}^2 + \omega_{zi}^2}$, defined as:

$$\sigma_{gyr} = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(\|\boldsymbol{\omega_i}\| - \bar{\omega})^2} \qquad (2)$$

With $\bar{\omega} = \frac{1}{N}\sum_{i=1}^{N}\|\boldsymbol{\omega_i}\|$

• **phone_handled**: Indicator that separates handled from mounted/stationary phone states, derived from a composite threshold test on σ_{acc} and σ_{gyr}, as outlined below.

The method exploits the temporal variability of the smartphone's tri-axial accelerometer and gyroscope signals, quantified through their standard deviations over consecutive temporal windows

A key stage of the detection pipeline is determining whether the smartphone is currently being held or manipulated by the user. We formulate this as a ternary classification problem with output variable *phone_handled* ∈ {0, 1, UNCERTAIN}, where 0 indicates that the device is stationary and unattended, 1 indicates active handling, and UNCERTAIN denotes that the available sensor evidence is insufficient to confidently assign either label. The introduction of an explicit UNCERTAIN state is a deliberate design choice: IMU readings acquired from a pocket, a bag, or a loosely resting device can exhibit sporadic motion artifacts that mimic the onset of handling, and forcing a binary decision in such cases degrades the precision of downstream phone-use inference.

The detector produces three possible outcomes: 1, indicating that the phone is held by the driver; 0, indicating that the phone is positioned on the vehicle cradle; and UNKNOWN, indicating insufficient or conflicting inertial evidence to reliably distinguish the two states.

Next, we normalize each raw statistic against a stationary/mounted baseline before combining them by defining:

$$\widehat{\sigma_{acc}} = \frac{\sigma_{acc} - \mu_{acc,0}}{\sigma_{acc,0}} \qquad (3)$$

$$\widehat{\sigma_{gyr}} = \frac{\sigma_{gyr} - \mu_{gyr,0}}{\sigma_{gyr,0}} \qquad (4)$$

Where $(\mu_{acc,0}, \sigma_{acc,0})$ and $(\mu_{gyr,0}, \sigma_{gyr,0})$ denote the mean and standard deviation of acceleration and angular velocity observed while the smartphone is known to be under the not handled (stationary) condition. Equations (3-4) remove device and sensor-scale dependance and they are based on the same rationale adopted for Z-score-based sensor fusion in wearable sensing literature.

The combined score:

$$s = \widehat{\sigma_{acc}} + \widehat{\sigma_{gyr}} \qquad (5)$$

After z-normalization, both quantities are dimensionless scores with zero mean and unit variance under their respective class-conditional distributions (stationary or handheld). The z-standardization step places both sensors on a common statistical scale, rendering them directly comparable in magnitude. Assigning equal weight to two commensurable, standardized features is the common principled choice in statistical pattern.

Next, rather than forcing a binary decision when the two sensor modalities provide conflicting or borderline evidence, the classifier employs a three-state decision rule:

$$phone_handled = \begin{cases} 0 & if\ s < \tau_L \\ UNKNOWN & if\ \tau_L \leq s < \tau_H \\ 1 & if\ s \geq \tau_H \end{cases} \qquad (6)$$

The *UNKNOWN* state is deliberately introduced as a rejection region around the decision boundary. Thus, windows exhibiting weak sensor excitation, contradictory accelerometer/gyroscope evidence, insufficient valid samples, or feature values falling within the empirically determined ambiguity region are not forced into either class. This conservative decision strategy is particularly important because low sensor variance does not necessarily imply that a phone is not being held, while elevated variance can also arise from external motion.

The two decision thresholds τ_L and τ_H (with $\tau_L < \tau_H$) are empirically estimated from a labeled *calibration corpus* collected prior to deployment. The corpus consists of two disjoint subsets of one-second windows: a *stationary set* of K_0 windows recorded while the phone rests untouched on a stable surface, and a *handled set* of K_1 windows recorded while the user actively manipulates the device. For each calibration window k, the combined z-normalized score is computed identically to the operational pipeline (equation 5)

This yields two empirical distributions: the stationary scores (k = 1, ... , K_0) and the handled scores (k = 1, ... , K_1). The lower threshold τ_L is defined as the 95th percentile of the stationary distribution:

$$\tau_L = Q_{0.95}\left(\{s_k^L\}_{k=1}^{K_0}\right) \qquad (7)$$

where $Q_{0.95}$ denotes the empirical quantile function at probability level 0.95. Intuitively, τ_L is placed at the upper tail of the stationary distribution so that 95% of genuinely unattended windows produce a score below this threshold and are correctly classified as 0 (not handled). The remaining 5% of stationary windows, those whose sensor readings happen to exhibit noise spikes or micro-vibrations that temporarily elevate s, fall above τ_L

and enter the uncertainty band rather than being falsely declared as handled. The choice of the 95th percentile (i.e., a 5% false-positive tolerance for the stationary class) controls the trade-off between the false-positive rate on the stationary class and the width of the uncertainty band: a higher percentile (e.g., 99th) would reduce stationary false positives but widen the band, deferring more genuine handling detections to the UNKNOWN state.

The upper threshold τ_H is defined symmetrically from the handled distribution, but at the lower tail:

$$\tau_H = Q_{0.05}\left(\{s_k^H\}_{k=1}^{K_1}\right) \quad (8)$$

Here $Q_{0.05}$ is the 5th percentile of the handled score distribution. This placement ensures that 95% of genuinely handled windows exceed τ_H and are correctly classified as 1. The 5% of handled windows that fall below τ_H correspond to brief pauses or lulls in handling activity where the IMU excitation momentarily drops near the stationary level; rather than forcing an abrupt label change, these windows are assigned to the UNKNOWN band. The symmetry of the two percentile levels (5% and 95%) is deliberate: it imposes an equal misclassification budget on both the stationary and handled classes, ensuring that neither class is systematically disadvantaged by the threshold placement.

The region between the two thresholds defines the *uncertainty band*:

$$[\tau_L, \tau_H] = [Q_{0.95}(s|stat.), Q_{0.05}(s|hand.)\,] \quad (9)$$

Any window whose combined score s falls within this interval is assigned the label UNKNOWN. The width of the band, $\tau_H - \tau_L$, is determined entirely by the degree of overlap between the two empirical score distributions. When the sensor features are highly discriminative (e.g., the phone is firmly held versus placed on a stable desk), the stationary and handled distributions are well-separated, $\tau_L < \tau_H$ holds easily, and the band is narrow, yielding high classification rates with few UNKNOWN outputs. When the two distributions overlap significantly (e.g., the phone is loosely held or subjected to environmental vibrations), the band widens, and a larger proportion of windows are conservatively flagged as UNCERTAIN. This adaptive behavior is a direct consequence of the data-driven percentile calibration: the thresholds automatically adjust to the sensor characteristics of the specific device and the user's individual handling style without requiring manual tuning.

Our calibration protocol proceeds as follows. During an initial calibration session, we place the phone on a stationary surface for approximately 300 seconds, yielding $K_0 \approx 60$ five-second windows for the stationary distribution. The user then handles the phone naturally (reading, scrolling, typing) for another 300 seconds, yielding $K_1 \approx 60$ windows for the handled distribution. For each window, σ_{acc} and σ_{gyr} are computed via Eqs. (1)–(2) of the companion section, z-normalized using the mean and standard deviation of the stationary calibration set (Eqs. 3-4) , and summed per Eq. (5) above. The two threshold values are then obtained by sorting each distribution and reading off the corresponding

percentiles. The entire calibration requires no labeled data beyond the binary “stationary versus handled” distinction and can be performed in under three minutes per user. If the calibration data are subsequently found to produce $\tau_L \geq \tau_H$ (i.e., the distributions overlap so heavily that no discriminative band exists), this indicates that the sensor data are insufficiently informative for the given device or environment, and the system should default to UNKNOWN for all windows until more favorable conditions are observed.

4.2.2.2 Smartphone Application-Level Behavioral Features

Smartphone operating systems provide behavioral interaction signals via the AccessibilityService on Android and the ScreenTime API on iOS 16+. However, because the proposed DT is optimized for the Android ecosystem, it leverages the AccessibilityService, which offers the necessary temporal granularity for extracting tap_rate, scroll_count, and is_typing. In contrast, the iOS ScreenTime API imposes stricter limitations: it does not expose tap_rate, and real-time observation of is_typing requires explicit user consent.

Through the application-level monitoring subsystem, we extract the following primary mobile device interaction features:

• **tap_rate** (continuous): The frequency of touch events per second within a given time window. Elevated values indicate active tactile interaction (e.g., typing a message).

• **scroll_count** (discrete): The number of scroll or fling events detected within the window, primarily indicative of passive content consumption (e.g., browsing social media).

• is_typing $\in \{0, 1\}$: A binary indicator derived from keyboard state monitoring, equal to 1 when a software keyboard is active in the foreground.

• **on_call** $\in \{0, 1\}$: A binary indicator equal to 1 during an active voice call (off-hook state), as reported by the Android TelephonyManager or iOS CallKit framework.

• **app_cat** (categorical): The category of the foreground application (e.g., NAVIGATION, MESSAGING, SOCIAL_MEDIA, PHONE, or OTHER), determined by mapping the application’s package name (Android) or bundle identifier (iOS) to a predefined taxonomy.

Additionally, we derive three composite binary features to characterize the nature of the interaction:

• **is_scrolling** $\in \{0, 1\}$: Equal to 1 if scroll_count > 0.

• **is_tapping** $\in \{0, 1\}$: Equal to 1 if tap_rate > τ_tap, where τ_tap = 1.5 taps/second.

• **is_active** $\in \{0, 1\}$: Equal to 1 if the user is actively manipulating the device, defined as the logical OR of phone_handled, is_typing, on_call, is_tapping, and is_scrolling. This distinguishes active physical interaction from passive screen viewing.

The complete feature set utilized in this study is formulated as two distinct vectors:

Primary Feature Vector (F, 9-dimensional): $\vec{F}$ = [v_mean, σ_acc, σ_gyr, phone_handled, tap_rate, scroll_count, is_typing, on_call, app_cat]

Composite Feature Vector (C ∈ {0, 1}^4): $\vec{C}$ = [is_moving, is_scrolling, is_tapping, is_active]

4.3. Decision Tree Algorithm

This section details the structure and sequential logic of the proposed decision tree (DT). We outline the core design principles, describe the topology and logic of the eight decision layers, provide formal pseudocode, and define the severity-calibration rule.

The proposed DT encodes domain knowledge regarding driver distraction into a hierarchy of threshold-based rules. Its primary advantages are full interpretability, since every decision maps directly to a specific rule, and the elimination of training data requirements, which allows for immediate deployment. The tree is explicitly designed to minimize false alarms while maintaining high sensitivity to genuine distraction events.

4.3.1 Decision Tree Design Principles

The DT architecture is governed by three core design principles:

• **Stationary Vehicle Exemption**: As the highest-priority rule, no alert is issued when the vehicle is stationary (*is_moving* = 0), regardless of phone interaction. Because a stationary vehicle poses no kinetic crash risk, this principle inherently eliminates false alarms during parked scenarios, at drive-throughs, or in standstill traffic.

• **Navigation App Exemption**: No alert is issued if a navigation application is in the foreground and the phone is not being physically handled (e.g., the device is mounted in a cradle). This exempts the most common legitimate use case for a smartphone while driving. However, if the phone is physically handled while a navigation app is active, the system proceeds to evaluate distraction risk, as the driver may be interacting with the device in a non-navigational manner.

• **Speed-Calibrated Risk**: Distraction severity is dynamically calibrated to vehicle speed, as higher speeds reduce the time and distance available to react to hazards while exponentially increasing crash energy. This aligns with Nilsson's Power Model (Nilsson, 2004), which demonstrates that the frequency of severe and fatal crashes scales with the second, third, and fourth powers of mean speed, respectively. Furthermore, as noted by Aarts and van Schagen (2006), higher speeds critically degrade a driver's ability to detect and respond to unexpected events. Consequently, the base severity assigned by the DT's primary rules is escalated using a speed-dependent multiplier (shown in Table 1): adding

zero tiers at Low speed, +1 tier at Medium speed, and +2 tiers at High speed, with the final severity capped at High.

Table 1. Speed-Risk Calibration Tier Table

Alert severity tier	Speed range (km/h)	Road context	DT escalation applied
(Low , Moderate, High)	$5 \leq v < 30$	Residential/school zones	0 (no escalation)
	$30 \leq v < 60$	Urban arterial/ suburban	+ 1 severity tier (cap at high)
	$v \geq 60$	Urban expressway/ highway	+ 2 severity tiers (cap at high)

• **Distraction-Type Differentiation and Activity Precedence**: The four distraction types are evaluated according to a strict priority sequence: texting, calling, tapping, and scrolling. This precedence ensures that the most cognitively and visually demanding interactions are identified first.

• **Fail-Safe Default**: If the vehicle is moving and *phone_handled* = 1, but no specific distraction condition is matched, a fallback rule triggers a Moderate severity alert. This ensures that no genuinely handled-phone event escapes detection, operating on the premise that over-alerting in ambiguous conditions is safer than missing a real distraction event.

• **Temporal Hysteresis**: To suppress transient false alarms caused by brief, sub-five-second glances, an alert is emitted only after three consecutive five-second windows agree on the same severity tier.

4.3.2. Decision Tree Structure and Sequence Logic

As illustrated in Figure 3, the DT is organized into eight layers evaluated in a top-down manner. The topology is structured so that exemption rules are evaluated first to suppress false alarms, followed by the most discriminative interaction tests.

• **L1 — Vehicle Motion Gate**: If *is_moving* = 0, the algorithm immediately returns no alert. Serving as the strongest false-alarm-suppression rule, it is evaluated first due to its minimal computational overhead.

• **L2 — Navigation Exemption**: A phone mounted in a cradle running a navigation application represents a canonical benign scenario. If *app_cat* = *NAVIGATION* AND *phone_handled* = 0, no alert is issued. Both conditions must be met simultaneously; if the phone is handled while a navigation app is in the foreground, the algorithm falls through to L3 for standard processing.

• **L3 — Phone Usage Branch**: If *is_active* = 1, active phone use is confirmed (regardless of whether the device is physically handled), and the algorithm proceeds to L4.

• **L4 — Typing Branch**: If phone activity is detected, the algorithm first tests *is_typing*. Texting constitutes the highest-severity distraction under both handheld and cradle-

mounted conditions. This is because texting requires simultaneous visual attention (reading the screen), manual engagement (keyboard interaction), and high cognitive load (message formulation).

• **L5 — On-Call Branch**: If typing is not detected, the algorithm evaluates *on_call*. A handheld call is assigned High severity, as it imposes sustained cognitive load, removes one hand from the steering wheel, and introduces manual distraction, regardless of whether the phone is held to the ear or used on speaker. A cradled call is assigned Moderate severity; while the driver's hands remain free, the cognitive load of the conversation still impairs hazard perception. If the handling state is *UNKNOWN*, the system assigns a Moderate severity alert as a conservative measure.

• **L6 — Scrolling, Tapping, and Fallback Branch**: If *is_scrolling* = 1 OR is_tapping = 1, the algorithm checks for active tapping. Tapping indicates visual-manual engagement of moderate duration, though without the sustained gaze dwell characteristic of typing. Both handheld and cradled tapping are assigned Moderate severity. If active tapping is not established, the algorithm concludes passive scrolling is occurring and assigns a Low severity flag. (Note: If both tapping and scrolling occur simultaneously, the algorithm defaults to the higher-severity tapping event). Finally, this layer implements a catch-all fallback: if *phone_handled* = 1, the vehicle is moving, and no previous conditions matched, the system triggers an *UNSPECIFIED HANDLING* condition with Moderate severity.

• **L7 — Speed-Based Risk Escalation**: After a base severity is assigned by Layers L1 through L6, the algorithm consults *v_mean* and escalates the severity according to Table 1: adding zero tiers at Low speed, +1 tier at Medium speed, and +2 tiers at High speed, with the final result capped at High.

• **L8 — Temporal Alert Persistence (Hysteresis)**: The system generates an alert only after N_{on} = 3 consecutive 5-second windows agree on the same severity tier, effectively suppressing transient single-window false alarms. Conversely, the alert is cleared only after N_{off} = 3 consecutive windows have passed without an alert condition.

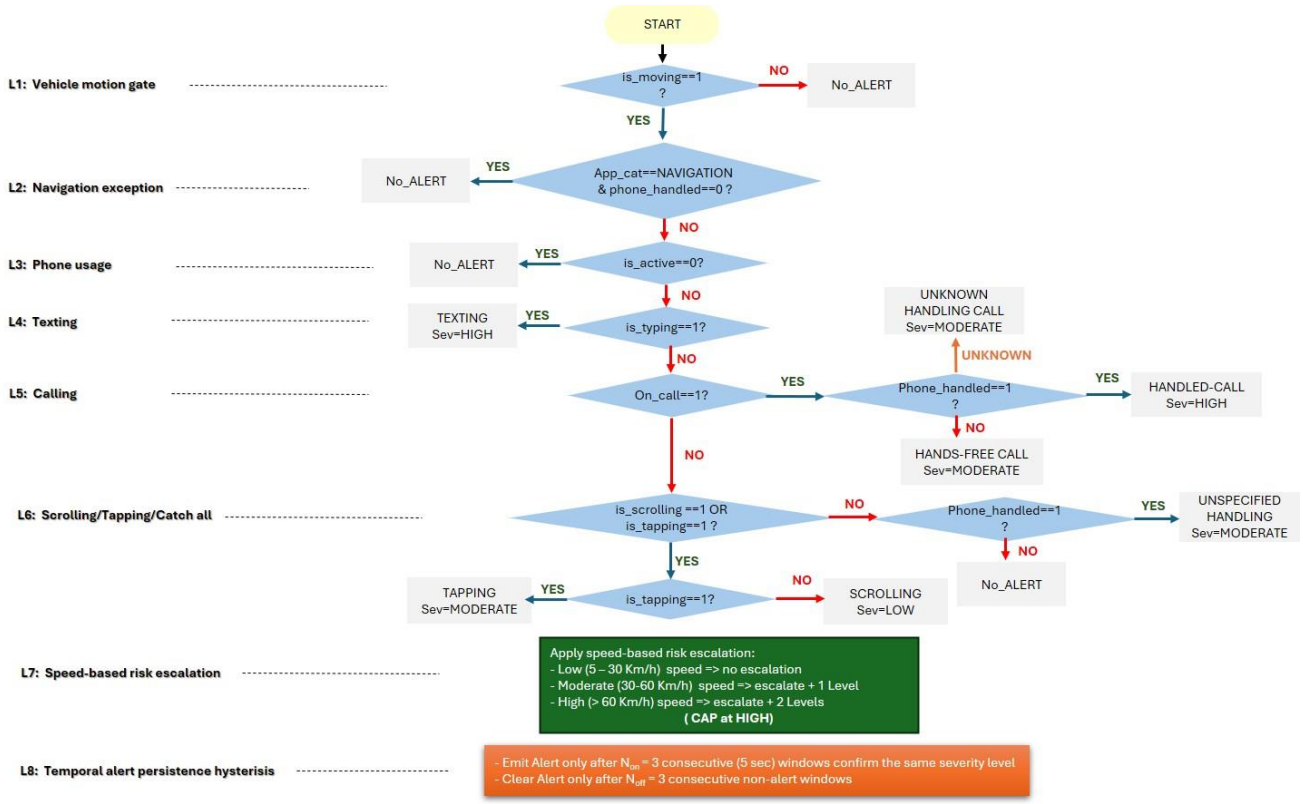


Figure 3. Decision Tree Layered Structure

4.3.3. Decision Tree Pseudocode

Figure 4 illustrates the decision tree algorithm as a sequence of prioritized rules:

• **Rule 0 (Stationary Exemption):** Serving as the highest-priority rule and the primary false-alarm mitigation mechanism, this rule explicitly exempts any phone interaction that occurs while the vehicle is stationary.

• **Rule 1 (Navigation Exemption)**: This rule addresses hands-free navigation, recognizing it as a legitimate, low-risk use case that does not require an alert.

• **Rule 2 (Inactivity Gate):** This rule ensures that no alert is generated if the phone is not in an active interaction state.

• **Rule 3 (Typing Detection):** This rule evaluates texting, which is classified as the highest-severity distraction type.

• **Rule 4 (Calling Detection):** This rule evaluates calling activities and assigns an appropriate severity level based on the device's physical handling state (handled, non-handled, or unknown). For example, if the phone is connected to the vehicle's Bluetooth system and remains unhandled (phone_handled = 0), the call is classified as hands-free. In this specific scenario, a Moderate base severity alert is emitted to account for the cognitive load associated with the conversation, despite the absence of manual distraction.

• **Rule 5 (Scrolling, Tapping, and Fallback):** This rule tests for scrolling and tapping activities. It also implements a conservative fallback mechanism: if the phone is being handled but the specific activity does not match any of the four defined categories, it is classified as a Moderate-risk distraction to ensure no potentially dangerous behavior escapes detection.

• **Rules 6 and 7 (Escalation and Hysteresis)**: These final rules implement the speed-based risk escalation and temporal alert persistence mechanisms, respectively, as detailed in the previous section.

Knowledge driven multimodal framework for context aware-mobile phone distraction monitoring

Input: Feature vector $\vec{F}$ = [v_mean, σ_{acc}, σ_{gyr}, phone_handled (0, 1, unknown), tap_rate, scroll_count, is_typing (0/1), on_call (0/1), app_cat]

Composite binary features: is_moving; is_active; is_scrolling; is_tapping

Definition of composite binary features:

is_moving=1 if v_mean > 5 km/h;

is_active=1 if (phone_handled=1 or is_typing=1 or on_call=1 or tap_rate > τ_{tap} or scroll_count > 0)

is scrolling_= 1 if scroll_count >0

is_tapping=1 if tap_rate > τ_{tap}

Parameters: v_min= 5Km/h; τ_{tap}= 1.5 taps/sec; σ_{acc}; σ_{gyr}; v_low= 30Km/h; v_high= 60Km/h; N_{on}=3; N_{off}=3

Output:

- Alert base category $C_{s_base} \in$ {NO_ALERT, LOW, MODERATE, HIGH}
- Alert risk-adjusted category $C_{s_adjusted} \in$ {LOW, MODERATE, HIGH}
- Interaction category $C_{interaction} \in$ {TEXTING, HANDLED_CALL, HANDS_FREE CALL, UNKNOWN_HANDLING CALL, TAPPING, SCROLLING, UNSPECIFIED HANDLING}
- Foreground app category $C_{app} \in$ {NAVIGATION, MESSAGING, PHONE_CALL, SOCIAL_MEDIA, WEB_BROWSING, VIDEO, AUDIO, UNKNOWN, …, OTHER}

1: **Rule 0 (Stationary vehicle): if** is_moving = 0 **then return** NO_ALERT // vehicle not moving
2: **Rule 1 (Exception: Hands-free navigation): if** app_cat = NAVIGATION and phone_handled = 0 **then return** NO_ALERT
3: **Rule 2 (Phone Usage): if** is_active = 0 **then return** NO_ALERT // phone not in use
4: **Rule 3 (Texting): if** is_typing = 1 **then**
5: $C_{interaction} \leftarrow$ TEXTING ; Cs_base $\leftarrow$ HIGH; $C_{app} \leftarrow$ app_cat
6: **Rule 4 (Calling): else if** on_call =1 **then**
7: **if** phone_handled = 1 **then**
8: $C_{interaction} \leftarrow$ HANDLED_CALL; $C_{s_base} \leftarrow$ HIGH; $C_{app} \leftarrow$ app_cat
9: **else if** phone_handled = 0 **then**
10: $C_{interaction} \leftarrow$ HANDS_FREE_CALL; $C_{s_base} \leftarrow$ MODERATE; $C_{app} \leftarrow$ app_cat
11: **else if** phone_handled = UNKNOWN **then**
12: $C_{interaction} \leftarrow$ UNKNOWN_HANDLING_CALL; $C_{s_base} \leftarrow$ MODERATE;
$C_{app} \leftarrow$ app_cat
13: **end if**

14: **Rule 5 (Scrolling/tapping/catch all)**
else if is_scrolling = 1 **or** is_tapping = 1 **then**
15: **if** is_tapping = 1 **then**
16: $C_{interaction} \leftarrow$ TAPPING; $C_{s_base} \leftarrow$ MODERATE; $C_{app} \leftarrow$ app_cat
17: **else**
18: $C_{interaction} \leftarrow$ SCROLLING; $C_{s_base} \leftarrow$ LOW; $C_{app} \leftarrow$ app_cat
19: **end if**
20: **else if** phone_handled =1 **then**
21: $C_{interaction} \leftarrow$ UNSPECIFIED_HANDLING; $C_{s_base} \leftarrow$ MODERATE; $C_{app} \leftarrow$ app_cat
22: **else return** NO ALERT
23: **end if**
24: **Rule 6 (Apply speed-based risk escalation) if** v_mean > v_high **then**
25: $C_{s_base} \leftarrow$ escalate (C_{s_base} , + 2) // cap at HIGH
26: **else if** v_mean > v_low **then**
27: $C_{s_base} \leftarrow$ escalate (C_{s_base} , + 1) // cap at HIGH
28: **end if**
29: **Rule 7 (temporal alert persistence / hysteresis)**
30: Emit alert only after N_{on}=3 consecutive windows confirmed same severity level
31: Clear alert only after N_{off} = 3 consecutive non-alert windows
32: Return : $C_{interaction}$, C_{s_base} , $C_{s_adjusted}$, C_{app}

Figure 4. High level per-window decision logic. Complexity *O*(1) per window

4.3.4. Alert Specification and Delivery Modalities

The system provides four distinct notification states (no alert, low severity, moderate severity, and high severity), determined by the vehicle's speed and the nature of the driver's phone interaction.

4.3.4.1. No Alert Condition

No alert is issued under any of the following conditions:

• The vehicle is stationary (*v_mean* ≤ 5 km/h), regardless of phone activity.

• A navigation application is in the foreground and the phone is not being actively handled.

• The phone is not engaged in any driver-initiated activity, regardless of its handling state.

These exemptions are critical for user adoption. A system that generates frequent false alarms is likely to be disabled by users, thereby entirely negating its intended safety benefits.

4.3.4.2. Low (Base) Severity Alert

A low-severity alert is triggered when only passive scrolling is detected, indicating a minor, non-immediate risk.

Alert Specification:

• Visual: Non-blocking banner notification with a cautionary message.

• Haptic: Brief vibration pulse (200 ms duration, medium intensity).

• Audio: None.

• Frequency: Repeated every 15 seconds while the distraction condition persists.

4.3.4.3. Moderate (Base) Severity Alert

A moderate-severity alert is triggered under any of the following conditions:

• A hands-free phone call is detected.

• Active tapping is detected.

• A phone call is detected, but the physical handling state is unknown.

• The phone is actively handled, but no specific distraction category is matched.

Alert Specification:

• Visual: Persistent notification bar displaying an amber warning icon and a brief description of the detected distraction.

• Haptic: Double vibration pulse (300 ms duration each, 100 ms gap, high intensity).

• Audio: Brief chime at moderate volume.

• Frequency: Repeated every 15 seconds while the distraction condition persists.

4.3.4.4. High (Base) Severity Alert

A high-severity alert is triggered under either of the following conditions:

• Texting activity is detected.

• A handheld phone call is detected.

Alert Specification:

• Visual: Persistent full-screen overlay displaying a prominent warning icon and instructional text.

• Haptic: Strong vibration pattern (500 ms pulse, repeated twice, high intensity).

• Audio: Audible alert tone at 80 dB.

• Frequency: Repeated every 15 seconds while the distraction condition persists.

4.3.5. Distinguishing Driver Smartphone from Passenger Smartphone

To prevent passenger phone usage from triggering false alarms, the system employs a simple device-classification framework. Upon the initiation of a driving trip (*is_moving* = 1), the system polls the smartphone's active Bluetooth connections. If the device maintains an active socket with the vehicle's OBD-II MAC address, it is classified as the driver device. Consequently, all other discovered Bluetooth peer devices within the vehicle's localized network are explicitly labeled as passenger devices and excluded from the distraction monitoring pipeline.

4.3.6. Mobile Application Design Considerations

The entire phone-use detection pipeline executes on the driver's smartphone as a background service. Upon startup, the application establishes a Bluetooth connection with the OBD-II adapter to continuously poll vehicle speed. Concurrently, it subscribes to inertial sensor event listeners and monitors the foreground application state via the operating system's accessibility and usage-statistics APIs. To function correctly, the mobile application requires the following runtime permissions:

• Bluetooth: For OBD-II adapter communication.

• Mobile Sensors: For accelerometer and gyroscope data access.

• Accessibility/Usage Access: For foreground application detection and touch event monitoring.

• Phone State: For incoming and outgoing call detection

4.3.6.1. Application Architecture

The mobile application employs a layered architecture comprising four distinct tiers: Presentation, Logic, Data, and Sensor. This separation of concerns allows each tier to be developed, tested, and updated independently. In addition, it isolates privacy-sensitive data handling within the Data Layer, ensuring strict control and audibility.

The **Sensor Layer** acquires raw data from hardware components, including the accelerometer, gyroscope, GPS, OBD-II interface, touchscreen, and application usage APIs. Each sensor is managed by a dedicated collector responsible for sampling rate configuration, data buffering, and error recovery. To guarantee continuous acquisition even when the application is backgrounded, these collectors operate as Android foreground services (or their iOS equivalents). Raw data is streamed into a circular feature buffer with a configurable window size (default: 5 seconds) to feed the Logic Layer.

The **Logic Layer** houses the core detection engine, which comprises the decision tree classifier, an alert manager, a session controller, and a report generator. The decision tree processes the feature vector from the Sensor Layer, while the alert manager

executes the escalation protocol, and the session controller manages the lifecycle of individual driving sessions.

The **Data Layer** handles persistent storage using an encrypted SQLite database. All session data, event logs, and configurations are stored locally on the device. This layer enforces strict data retention policies, automatically purging expired records and anonymizing aggregated statistics. By design, the application functions entirely offline, transmitting no data to external servers under standard operation.

The **Presentation Layer** provides the user interface, including the alert display (rendering visual warnings and dashboard indicators), a settings panel (for alert preferences, sensitivity, and privacy), a statistics dashboard (for distraction reports), and a consent manager. To avoid inducing further distraction, this layer is intentionally minimal; during driving, the default view displays only a small dashboard indicator, with detailed statistics accessible only when the vehicle is stationary.

4.3.6.2. Privacy Mechanisms

The following privacy mechanisms are implemented to mitigate risks associated with unauthorized access to smartphone sensor data and usage history:

• **On-Device Processing**: All computations execute locally within the application sandbox. No sensor data, feature vectors, risk levels, or alert events are transmitted to external servers. This architecture eliminates interception risks and ensures continuous functionality in network-deprived environments.

• **Encryption and Secure Storage**: All data at rest is secured using AES-256 encryption, with keys derived from the device's hardware-backed keystore. This prevents data extraction in the event of device loss or theft. The encryption process is transparent to the Logic and Presentation layers, which interact with the database via a standard API.

• **Consent Management**: The application utilizes a granular consent management system. Upon first launch, users are presented with plain-language explanations of each data category (e.g., sensor data, touch events, speed data) and can toggle collection independently. If consent for a specific category is withdrawn, data collection ceases immediately, and existing records of that type are purged. Furthermore, an immutable audit log records the timestamp, affected category, and nature of all consent changes.

• **Data Retention and Automatic Purging**: Configurable retention policies strictly limit data lifespan. By default, event-level logs are purged after 30 days, aggregated statistics after 6 months, and session metadata after 12 months. Minimum retention limits are enforced to maintain statistical relevance. A background maintenance task executes this purging daily at a user-configured time (default: 3:00 AM local time).

4.3.6.3. System Design

The logging system records distraction events and state transitions to facilitate user awareness and enable retrospective safety analysis. It balances analytical completeness with privacy by intentionally omitting raw sensor readings and specific application names. Instead, each log entry captures a timestamp, an event type (e.g., ALERT_TRIGGERED, ALERT_ESCALATED, ALERT_DE-ESCALATED, ALERT_DISMISSED, SESSION_START, SESSION_END, RISK_LEVEL_UPDATE), the current alert level, the distraction type, and a subset of the derived feature vector. This provides sufficient context to reconstruct the detection logic without exposing personally identifiable information.

4.3.6.4. Statistical Reports and Awareness Enhancement

To foster behavioral modification, the application translates detection data into statistical reports that visualize a driver's distraction patterns over time. The system generates daily, weekly, and monthly summaries detailing distraction frequency, total duration, and peak risk levels. Weekly reports include trend lines comparing current performance to prior weeks, while monthly reports introduce long-term visualizations and a cumulative safety score. This feedback loop empowers drivers to correlate specific phone-use habits with elevated risk, thereby encouraging self-regulation.

4.3.7. Battery and Resource Optimization

Continuous sensor monitoring poses a significant barrier to user adoption due to battery drain. To mitigate this, the application dynamically adjusts sampling rates based on the driving state. When the vehicle is stationary (*v_moving* = 0), accelerometer and gyroscope sampling rates drop from 50 Hz to 10 Hz, and the speed polling interval increases from 1 second to 5 seconds. Because stationary periods constitute the majority of a device's active hours, this substantially reduces computational load. Furthermore, the decision tree is evaluated only when the feature buffer completes a full 5-second epoch, reducing classifier execution frequency from 50 times per second to just once every 5 seconds.

References

Nilsso. G. (2004). Traffic Safety Dimensions and the Power Model to Describe the Effect of Speed on Safety, Lund Institute of Technology and Society. Traffic Engineering.

Aarts, L., & Van Schagen, I. (2006). Driving speed and the risk of road crashes: A review. Accident Analysis and Prevention, 38(2), 215–224. https://doi.org/10.1016/j.aap.2005.07.004